Robot Móvil para rehabilitación

Luis Antonio Sánchez Monterde

Dedicado a mi familia

Agradecimientos

A mis amigos del pasado, del presente y del futuro.

Resumen

Una bonita historia

Índice general

Agradecimientos **V**
Resumen VII

Lista de figuras **XIII**

Lista de tablas **XV**

1. Introducción **1**

2. Objetivos **3**
2.1. Objetivo Principal 3
2.2. Objetivos Específicos 3

3. Metodología **5**
3.1. Dos es mejor que una 5
3.1.1. RUP 5
3.1.2. XP 7
3.2. ¿Porqué no usar una metodología clásica? 7

4. Trazadora. Primera iteración **9**
4.1. Requerimientos 10
4.1.1. El movimiento 10
4.1.2. Lenguaje de programación 11
4.2. Análisis 11
4.2.1. Selección de robot 11
4.3. Diseño 12
4.3.1. Arquitectura electrónica 12
4.3.2. Arquitectura de software 15

4.3.3. Arquitectura mecánica 17
4.4. Implementación 17
4.4.1. Implementación mecánica 19
4.4.2. Implementación electrónica 19
4.4.3. Implementación del programa 19
4.5. Pruebas 26
4.6. Objetivos refinados 27

5. Doble propulsor. Segunda iteración 29
5.1. Requerimientos 29
5.2. Análisis 29
5.3. Diseño 30
5.3.1. Arquitectura electrónica 30
5.3.2. Arquitectura mecánica 30
5.3.3. Arquitectura de software 30
5.4. Implementación 30
5.4.1. Implementación electrónica y mecánica 30
5.4.2. Implementación de software 30
5.5. Pruebas 35
5.6. Segunda iteración 35
5.6.1. Requerimientos y Análisis 35
5.6.2. Diseño 35
5.6.3. Implementación 36
5.6.4. Pruebas 36
5.7. Objetivos refinados 36

6. Volviendo al pasado, tercera iteración 41
6.1. Requerimientos 41
6.2. Análisis 41
6.3. Diseño 42
6.3.1. Diseño electrónico 42
6.3.2. Diseño de programa 43
6.4. Implementación 45
6.5. Pruebas 49
6.6. Objetivos refinados 49

7. Moverse en línea recta **51**
7.1. Requerimientos 52
7.2. Análisis 52
7.3. Diseño 52
7.3.1. Diseño del programa 53
7.4. Pruebas 60
7.5. Objetivos refinados 60

8. Un nuevo giro al proyecto **63**

Índice de figuras

4.1. Esquemático 1 13
4.2. Arquitectura de software 1.1 16
4.3. Arquitectura de software 1.2 17
4.4. Plano 1 18
4.5. Robot 1 20

5.1. Esquemático 2 31
5.2. Plano 2 32
5.3. Arquitectura de software 2 33
5.4. Esquemático 2.2 37
5.5. Arquitectura de software 2.2 38

6.1. Puente para motor de DC 42
6.2. Esquemático 3 44

7.1. PWM 52
7.2. Arquitectura de software 4 56

Índice de tablas

Capítulo 1

Introducción

Desafortunadamente hay muchas formas de perder la movilidad en brazos y manos, desde accidentes hasta degeneraciones neurológicas. Existen tratamientos para ayudar en la recuperación motriz, la intención es desarrollar un robot móvil que ayude en las terapias de recuperación. La función del robot consiste en mover los miembros superiores dañados en un secuencia de trayectorias, la repetición continua de ejercicios de movimiento ha demostrado ser de útilidad en la recuperación de movilidad.

Para lograrlo hago uso del la metodología RUP, Rational Unified Process [6], desarrollada en el mundo de software, por Ivar Jacobson, es una metodología del tipo plan driven o manejada por plan, aunque surgida del software puede ser aplicada también a otros ambientes, no se pretende hacer una implementación completa de RUP, se utilizara un pequeño subconjunto de las directivas que considero son suficientes para cumplir con las expectativas del proyecto. La idea es dividir el proyecto en una pequeña cascada de pasos y posteriormente repetir la cascada hasta que los objetivos se alcancen.

En concreto se realizara un proceso iterativo e incremental, agregando funcionalidad por funcionalidad e integrándolas paso a paso.

El objetivo clave es cumplir con las expectativas del paciente y medico y para hacerlo, tengo la certeza de que esta es la aproximación que dará el mejor resultado.

Capítulo 2

Objetivos

2.1. Objetivo Principal

- Analizar, diseñar e implementar un robot móvil para asistir en la recuperación de pacientes con inmovilidad en los brazos.

2.2. Objetivos Específicos

El objetivo principal es claro y muestra perfectamente las intensiones del presente trabajo, existen varios objetivos específicos, sin embargo he decidido que sean mutables a lo largo de la investigación y desarrollo, la razón no involucra ir acomodando las expectativas a mi beneficio si no exactamente lo contrario, al momento de iniciar no tengo todos los detalles y es ingenuo pensar que con el poco conocimiento que tengo sobre el problema puedo realizar la mejor solución, el lector quizá pensará que una investigación previa dará la información suficiente, pienso que esto no es así, la mejor forma de alcanzar todos los requisitos es obtener información del problema mismo, de esta forma es él quien nos guía sobre el desarrollo, derivando en un procesos de constante iteración y realimentación.

Los objetivos a continuación son los que considero importantes en este instante (en el inicio del presente trabajo), pero sé y adelanto que irán cambiando, otra forma de llamar a los objetivos es requerimientos, los usaré de forma indistinta.

- Para el paciente.

    1. Mover el robot de un punto A a B.

2. Potencia suficiente para soportar la carga.
3. Movimiento save.
4. Rotación libre.

- Para el médico.
  1. Interfaz de trayectorias.
- Para el desarrollador.
  1. Dominar un nuevo lenguaje de programación.
  2. Metodologías RUP y XP.

Capítulo 3

Metodología

La metodología elegida tiene un impacto fundamental en el desarrollo del proyecto, existen diferentes y variadas opciones, personalmente no fue hasta que comencé a indagar en este tema que me comencé a sentir seguro de mi trabajo, realizar cualquier proyecto era más cuestión de suerte y ganas que de puesta en practica del conocimiento. De las muchas opciones, me decanto por dos:

RUP Rational Unified Process.

XP Extreme Programming.

3.1. Dos es mejor que una

¿Se puede desarrollar un proyecto con dos metodologías?, la respuesta depende del tipo de proyecto, en este caso la respuesta es un contundente si, el proyecto es muy amplio implica partes mecánicas, eléctricas y de programación, mi propuesta es utilizar *RUP* para las partes mecánicas y eléctricas y *XP* para la programación.

3.1.1. RUP

Dentro de las metodologías hay dos polos, las ágiles y las guiadas por plan, RUP es una metodología guiada por plan, sin embargo no está en un extremo opuesto a las ágiles.

RUP consiste en una serie de pasos bien definidos, esta diseñada para abarcar proyectos muy grandes, proyectos que implican colaboraciones entre países o entre

empresas transnacionales, mi proyecto es pequeño en comparación, sin embargo considero adecuado aprovechar varias de las características de esta metodología, los siguientes escalones son el esqueleto del proceso de desarrollo:

Requisitación En este primer paso estamos en el peor de los mundos, pues tenemos poca información del problema y una especificación pobre. Hacer una toma de requisitos en la metodología RUP, implica entender el lenguaje del dominio, no es lo mismo hablar con un médico que con un agricultor o un chef de cocina, entender a quien tiene el problema en es un paso fundamental para poder diseñar una solución que satisfaga sus necesidades, no entender el lenguaje implicaría una serie de adivinanzas desafortunadas.

Cuando el entendimiento del lenguaje es suficiente, la prioridad es encontrar los requerimientos, nuevamente los requerimientos vienen de parte de quien requiere la solución.

Análisis Con los requerimientos definidos, procedemos a analizarlos, el proceso de análisis consiste en transformar los requerimientos del lenguaje del dominio a el lenguaje de la ingeniería, cada requerimiento puede abarcar varias áreas, para el robot móvil implica mecánica, electronica y programación.

Diseño Cada área de la ingeniería tiene una forma de diseño propia, en el caso de la mecánica y la electronica consiste en crear la arquitectura que satisfaga el requerimiento y posteriormente hacer los cálculos de los parámetros que la conforman. En el caso de la programación implica definir la arquitectura y encontrar las entidades que la satisfagan, la granularidad propia del software dificulta encontrar tales entidades.

Implemetación La implementación es el trabajo pesado y manual, aquí es donde se ensamblan los componentes, se alambran los circuitos y se escribe el código, es la etapa más lenta de todas, el punto de hacer una metodología es establecer las condiciones necesarias para que esta etapa pueda fluir lo más suave posible.

Pruebas Realizada la implementación se debe verificar que cumpla con las expectativas, si no lo hicieran debe repetirse el proceso nuevamente, no se trata de iniciar desde cero, si no de corregir lo que se erró en el proceso.

La intensión es realizar esta cascada de pasos para cada uno de los requisitos, las veces que sean necesarias hasta que se alcancen los mismos. En las primeras iteraciones la requisición se llevaran la mayor parte del tiempo, y con forme avancen las iteraciones irán perdiendo protagonismo pues cada vez estarán más refinados, en las ultimas iteraciones las pruebas se llevarán el protagonismo.

3.1.2. XP

La metodología *Extreme Programming* (XP), es una de las revolucionarias de los últimos tiempos, a diferencia de de RUP, donde se separan las fases de desarrollo, en XP el análisis, diseño, pruebas e implementación se van haciendo al mismo tiempo, XP es el padre de las modernas SCRUM y KANBAN entre muchas otras, está diseñada para proyectos medianos y pequeños, pero se requiere una habilidad técnica muy alta para llévala acabo además de que solo se recomienda para proyectos de software.

La técnica insignia de las metodologías ágiles es el diseño guiado por pruebas TDD[3], Test Drive Development en ingles, esta técnica consiste en escribir la prueba primero y después escribir solo el código necesario para que la prueba pase, la premisa del TDD es: *la mejor forma de corregir bugs es evitar introducirlos*. Al realizar las pruebas primero se obliga a escribir código correcto, gracias a las multiples herramientas existentes sobre pruebas unitarias, es posible automatizar el proceso, personalmente he adoptado el TDD y ya no concibo programar de otra forma, aunque reconozco que proceso fue traumático, tuve que literalmente olvidar lo aprendido y volver a aprender con nuevas directivas.

3.2. ¿Porqué no usar una metodología clásica?

Hay varias razones. El proceso de desarrollo de un proyecto según el PmBok es el siguiente

1. Inicio.- Definir el proyecto.
2. Planificación.- Se establecen los objetivos, se analizan y se diseña la solución.
3. Implementación
4. Medición de desempeño.
5. Cierre.

Ciertamente no difiere mucho de RUP, sin embargo no implica iteración en ningún momento, definir el proyecto al inicio es muy difícil cuando de proyectos innovadores se trata, lo más probable es que vamos a fallar, pienso que es más valioso aprender a lidiar con los cambios que planificar un proceso perfecto. El diseño temprano e implementación tardía implica una rigidez enorme, si se añadieran requisitos obligarían a rediseñar todo el proyecto. ¿Y la planificación?, ¿acaso no es

bueno planificar?, ami parecer planificar a corto plazo es bueno, planificar grandes cantidades de trabajo es un ejercicio inútil.

–Vamos a diseñar este proyecto y en dos meses el 5 de Junio comenzamos a implementarlo.–

Permítame reírme de sus planes.

¿Qué pasa si una vez implementado el proyecto no pasa las pruebas?, con suerte los cambios para pasar las pruebas serán lo suficientemente pequeños para poder desplegarlos, pero si no es así, si por alguna razón se nos escapo un detalle en el diseño o si el cliente añade una nueva característica, entonces sera el momento adecuado para hechar a andar nuestra imaginación para inventar las escusas más creativas, pues implementar los cambios requerirán un proceso lento y doloroso.

El mundo ideal no existe, la experiencia me indica que las cosas siempre cambian, una metodología flexible es la que decido utilizar.

Capítulo 4

Trazadora. Primera iteración

Una hoja en blanco es un mundo de posibilidades para algunos, si embargo para mi es una situación intimidante y abrumadora, la solución es adquirir realimentación tan rápido como sea posible.

Un tipo de bala utilizada por los vehículos artillados es la llamada bala trazadora, tiene la característica de brillar de color rojo cuando es detonada y cuando se impacta produce un destello de luz, el objetivo de esta bala es iluminar la trayectoria del resto de balas cuando son disparadas en ráfagas, Dave Hunt indica el uso de una bala trazadora,

"We use the term tracer bullet development to visually illustrate the need for immediate feedback under actual conditions with a moving goal."[4]

Una implementación no exhaustiva que abarca todas las capas de desarrollo, el objetivo es el mismo que la trazadora real, iluminar cada una de las capas de diseño del problema, con suerte podremos ver característica que no aparecen a simple vista.

En esta sección expongo la primer iteración correspondiente a una bala trazadora, recalco que esta iteración busca realimentación del problema más que satisfacer un requerimiento.

4.1. Requerimientos

4.1.1. El movimiento

Para mi fortuna tengo la asistencia del laboratorio del *Laboratorio de Investigación y Desarrollo de Aplicaciones Interactivas para la Neuro-Rehabilitación* del Instituto de Fisiología Celular de la UNAM, quienes son expertos y tienen gran experiencia tratando y rehabilitando a este tipo de pacientes, la mayor parte del entendimiento y realimentación es gracias a ellos.

Es aquí donde los objetivos específicos se definen, no antes, cada iteración los ira refinando.

Una forma simple de adquirir los requerimientos es por medio de casos de uso,

"A use case is all the ways of using a system to achieve a particular goal for a particular user."[7]

Un *actor* es aquel que requiere un caso de uso, para el proyecto un actor inmediato es el paciente, pues es quien interactúa con el dispositivo de manera directa y durante la mayor parte del tiempo, sin embargo no es el único, otro actor muy importante es el operador, usualmente el operador es el médico o enfermera, él también requiere usar el dispositivo y tiene expectativas en él. Un tercer actor es el desarrollador, el impone sus caprichos igualmente. Por ahora estos son los actores identificados, pero no descarto una anexión o eliminación en el futuro. Es necesario mencionar que un actor puede no ser una persona, puede ser otro sistema o dispositivo.

Los requisitos sintetizados en esta etapa ya se mencionaron en el capitulo 2, se actualizaran al final de la iteración. El primero objetivo/caso de uso es

- Mover de un punto A a B.

Nuestro objetivo, mover de un punto A a un punto B, es muy amplio, implementar algo así llevaría mucho tiempo, nuestra prioridad por ahora es obtener información, así que acotare el objetivo a desplazar el robot hacia adelante nada más.

- Desplazar hacia adelante.

Considero que esta es una buena bala trazadora pues el núcleo del problema es mover la posición de un lado a otro.

4.1.2. Lenguaje de programación

Uno de los objetivos es dominar un nuevo lenguaje de programación, el lenguaje que lleva la batuta en el mundo del firmware es *C*, es un lenguaje sencillo, pequeño y con un soporte prácticamente universal, sin embargo tiene un problema, el tiempo de desarrollo siempre es mayor que si usáramos lenguajes de más alto nivel. *C++* es un lenguaje que poco a poco ha ganado terreno en el software embebido, tiene la ventaja de soportar los conceptos de programación orientada a objetos, por otro lado es un lenguaje muy complejo, muy grande, difícil de aprender, feo y todos lo odian.

La elección de lenguaje me permito hacerla a capricho, pues me considero un experto en el lenguaje *C* y uno de mis requisitos es dominar un nuevo lenguaje, que mejor oportunidad que esta, la elección es C++, aunque igualmente me hubiera gustado hacer la implementación en Rust o en Nim, lenguajes modernos que pretenden ser no solo una alternativa si no; un reemplazo para C y C++.

4.2. Análisis

Implementaremos el primer requisito.

Hay configuraciones de dispositivos que pudieran realizar esta tarea, considero que tomar una de ellas y adaptarla a nuestras necesidades es la opción correcta.

4.2.1. Selección de robot

Dentro de los dispositivos disponibles la elección era obvia, un robot, ¿pero de que tipo?, estas tres opciones suenan interesantes:

Robot antropomorfico sin duda es el robot más versátil que existe, es ampliamente utilizado en la industria manufacturera sobre todo en la automotriz, sin embargo tiene una gran desventaja para nuestro propósito, es muy costoso de implementar, principalmente por el tipo de actuadores que requiere, la configuración produce un torque muy grande en su base, lo que deriva en una base de acero y actuadores hidráulicos o neumáticos.

Robot prismático Este robot es excelente para movimientos precisos en el plano, es fácil de implementar y soporta cargas muy grandes, en un principio fue una opción muy atractiva, pero tiene un par de desventajas, es relativamente caro y es menos mantenible que la tercera opción.

Robot móvil Es la opción que satisface de mejor manera los requerimientos, se puede desplazar sobre trayectorias largas, rectas o circulares, es rápido, ligero y de los tres es el más barato, no tiene la precisión del robot prismático pero no la requerimos, pocas partes mecánicas y ampliamente documentado. En contraste es más difícil de programar pues requiere más sensores que los anteriores y sus acciones se limitan al plano horizontal.

La elección es un robot móvil, ¿con qué características?, con las necesarias, las que nos indique el problema para su resolución. Por ahora nos interesa trasladar el robot hacia adelante, requerimos

- Un propulsor.
- Una fuente de energía.
- Una rutina que haga el trabajo.
- Una base que acople todo.

4.3. Diseño

Esta sección se trata de pensar, se trata de ver el bosque antes de ver el árbol, decido dividir el diseño en tres partes, pues evidentemente cada una requiere su propio lenguaje y herramientas particulares.

4.3.1. Arquitectura electrónica

El esquemático se muestra a continuación en la figura 4.1.

Selección de propulsor

Lo evidente es que requerimos movimiento, para ello necesitamos controlar un dispositivo que pueda causarlo, opciones hay muchas, motores, turbinas, campos de fuerza, la elección obvia es un motor eléctrico, por el tamaño y el costo principalmente. ¿De que tipo?, por ahora un motor de corriente directa cualquiera nos sirve, más adelante los requerimientos nos indicaran el adecuado. Decido utilizar un motor de corriente directa reciclado de una vieja casetera. Desafortunadamente no cuento con las características técnicas de este dispositivo, no tiene marca ni leyenda de identificación, procedo a realizar una prueba de rotor libre.

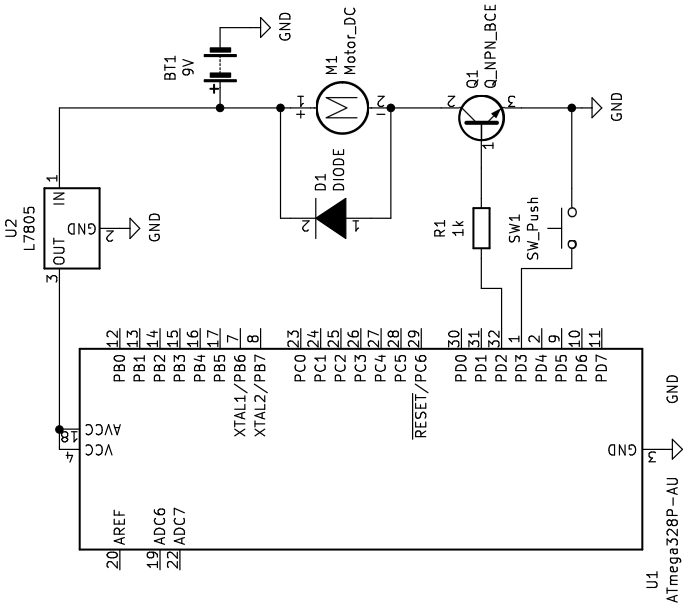

Figura 4.1: Esquemático 1

La prueba de rotor libre consiste en medir la corriente que consume el motor cuando no tiene carga mecánica, la medición es de tan solo 0.03 Ampers, con una tensión de 9 volts en sus terminales.

Repito la prueba pero con el robot bloqueado, con la misma tensión, la corriente sube dramáticamente hasta $500mA$.

Selección de dispositivo programable

Para controlar un motor hay varias opciones, hablando de dispositivos programables las opciones se reducen a dos una sbc (computadora en una sola tarjeta) y microcontroladores, aunque la sbc es mejor en prestaciones, es más cara y no es necesaria, existen microcontroladores suficientemente poderosos para nuestros objetivos.

De entre los muchos microcontroladores me interesa que tenga documentación y soporte, el atmega328p[1] del Arduino Uno suele ser la primera opción, pero dada la complejidad del proyecto me da la sensación de que la memoria programable puede no ser suficiente, no lo sé, comenzare con este dispositivo y si lo requiero lo cambio por uno con mayores prestaciones.

Selección de fuente de poder

El sistema requiere una fuente de alimentación, baterías son la elección adecuada, debido al tamaño y la autonomía, las agregaremos como requisitos y más adelante las especificaremos, por ahora cualquier batería con tensión mayor a 5 volts basta, me decido por una pila cuadrada de 9 volts.

Control del motor

Para que el motor pueda girar se requiere inducir una corriente eléctrica a través de sus terminales, cada pin del microcontrolador solo tiene la capacidad de entregar $20[mA]$ como máximo, no podemos conectar el motor a un pin directamente. La solución para un motor que va en un solo sentido, es un transistor de propósito general que soporte más de $500[mA]$ por su colector. Un 2n2222A[2] es suficiente.

El transistor se utiliza como switch, se induce una pequeña corriente a través de la base del transistor, esto provoca que el colector "se abra" y pueda conducir una corriente proporcional a la base, de 100 veces o más para este transistor, si se deja de inducir corriente por la base, el transistor "se cierra" y el colector deja de conducir corriente. La forma de inducir la corriente es muy sencilla, se conecta una resistencia entre el pin de la base y el pin del microcontrolador como indica el

esquemático. Con ley de Ohm:

$$i_b = \frac{V_{pin} - V_{be}}{R_b}$$

despejando la resistencia

$$R_b = \frac{V_{pin} - V_{be}}{i_b} = \frac{5 - 0.7}{5m}$$

$$R_b = 860\Omega$$

Podemos permitirnos ser conservadores y poner una resistencia de 1000Ω.

4.3.2. Arquitectura de software

Soy electrónico pero soy una persona de software, estás etapas de diseño son muy satisfactorias para mí.

Los planos arquitectónicos de una casa deben verse como los de una casa y los de una escuela como una escuela, seria absurdo una casa con fachada de escuela, desafortunadamente la gente de software no alcanza a entender bien esta idea y programan todo de la misma forma, no es mi caso, mi arquitectura se verá como un robot móvil.

Me gusta utilizar la arquitectura definida por Robert C. Martin *Clean Architecture* [8], es un modelo basado en capas (coo una cebolla), donde la capa más profunda es constituida por las llamadas entidades criticas, las capas intermedias por las reglas de negocio y las capas exteriores las componen los detalles.

Entidades criticas Son estructuras de datos u objetos que solo contiene datos y no funcionalidad. Tienen una característica muy descriptiva, lejos de ser abstractas son muy concretas y cambian muy poco, es una buena idea depender de entes concretos, es decir todos las entidades deben depender de las entidades criticas, queda prohibido que las entidades criticas dependendan de los detalles.

Reglas de negocio Mejor conocidas como business rules, son funcionalidades que modifican y/o acceden a las entidades criticas para satisfacer los requerimientos. Nuestro robot requiere avanzar, usaremos software para controlar el avance, sin embargo el software no es necesario, bien podríamos empujar el robot, o hacer que un caballo lo jale, hay muchas formas de hacerlo avanzar, a eso me refiero con regla de negocio, a un procedimiento que realiza el dispositivo no importando si esta automatizado por software o no.

Detalles Todo aquello fuera del negocio, generalmente periféricos. Los detalles no tienen que ver con el negocio, son simples elecciones a capricho o a conveniencia, por ejemplo, el tipo de motor, al robot no le interesa que tipo de motor hay, tan solo le interesa que al enviarle un comando el motor giré y provoque el avance.

Requerimos mover el robot, a continuación enumero el bosque:

1. Por ahora solo se requiere ir hacia adelante, un solo pin de propósito general basta para lograrlo.
2. No hay requerimientos de distancia, así que un temporizador sera suficiente.
3. Un pulsador conectado a un pin para iniciar la secuencia.

Por ahora pienso que es suficiente, el diagrama de bloques de la arquitectura se muestra en la figura4.2.

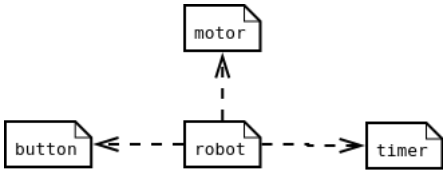

Figura 4.2: Arquitectura de software 1.1

Robot es una entidad que depende de timer, motor y button, acabo de mencionar que esta prohibido que capas centrales dependan de capas externas, la forma de arreglan esto es utilizar interfaces.

Una interfaz es una clase que no tiene parámetros, solo métodos, y los métodos son del tipo virtual[1].

No se pueden crear instancias/objetos de interfaces, ¿entonces para que y como se usan?, la respuesta no es trivial, su principal objetivo es desacoplar módulos/clases, sin embargo también sirven para extender clases y también es posible revertir dependencias. Dada una interfaz, por ejemplo Motor, se pueden implementar nuevas clases derivadas de motor, MotorDC, MotorAC, etc, al ser clases derivadas siguen siendo Motor y contienen la misma funcionalidad, el como lo logran es un detalle, pero su *cara* hacia el mundo es la misma. Dicho esto, se propone la arquitectura de la figura 4.3. Ahora robot ya no depende de los detalles, los detalles dependen de las reglas de negocio.

[1]Un método virtual tiene la características de poder ser redefinido por las clases heredadas

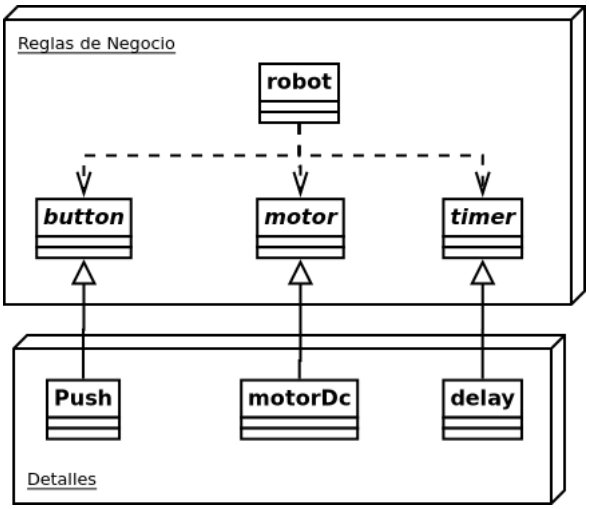

Figura 4.3: Arquitectura de software 1.2

4.3.3. Arquitectura mecánica

Por último pero no al final, la arquitectura mecánica, a diferencia de la electrónica y la programación la parte mecánica tiene un contacto directo con el paciente, es necesario poner suficiente esfuerzo pues de ella dependerá en gran proporción la satisfacción del paciente.

El diagrama es sencillo en esta etapa, solo requerimos una superficie que soporte todos los componentes, propongo el esquema mostrado en la figura 4.4, Consiste de una base circular soportada por 4 pequeños postes y exactamente a la mitad un motor, por supuesto la mayor parte de esto cambiara con conforme el desarrollo avance.

4.4. Implementación

Las tres partes pueden implementarse en forma individual y paralela, al final se hará una etapa de integración.[2] Por simple gusto implemento primero la parte mecánica.

[2]En esta etapa es donde se diferencian los hombres de los niños.

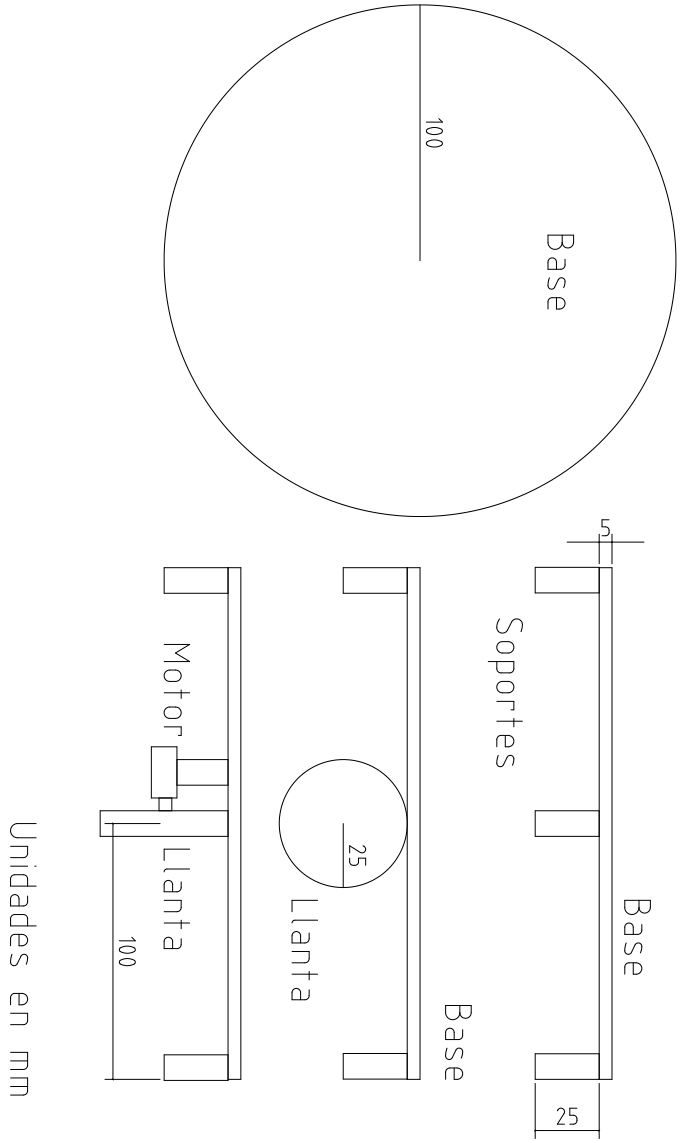

Figura 4.4: Plano 1

4.4.1. Implementación mecánica

Cualquier material nos sirve para hacer la base, me decido por utilizar cartón, es fácil de manipular y barato, para los soportes utilizo lapices, la llanta es reciclada de un juguete y el motor también es reciclado.

El resultado es un juguete muy feo, pero me da las sensaciones que estaba buscando figura 4.5.

4.4.2. Implementación electrónica

Para facilitar la implementación utilizo un Arduino nano, tiene incorporado un regulador de tensión de $5V$, no tendré que preocuparme por regular por ahora, tan solo se realiza el cableado indicado en el esquemático en una tablilla de prototipos.

4.4.3. Implementación del programa

La implementación mecánica y eléctrica fue sencilla, pero la programación requiere más tiempo y cuidado, sobre todo la primera vez.

Es muy complicado explicar el proceso de programación, la forma correcta sería colocar el código y su evolución con cada una de las decisiones de diseño que tomamos, pero para un programa de 10 lineas, tendría que mostrar unas 20 versiones, pasando por cada una de las pruebas unitarias y analizando cada uno de los cambios. Prefiero mostrar únicamente las partes que considero relevantes.

Robot

Aprovechando los conceptos de modularidad y programación orientada a objetos creamos una clase robot, por agregación tiene tres apuntadores, son los objetos que utiliza el robot, su constructor[3] y por supuesto la acción que nos interesa, moverse hacia adelante. Los $\#include$ contienen la declaración de las interfaces.

```
1  #ifndef ROBOT_H
2  #define ROBOT_H
3
4  #include "motor.h"
5  #include "timer.h"
6  #include "button.h"
7
```

[3] El constructor de una clase es un método que se ejecuta automáticamente al crear una instancia de la clase

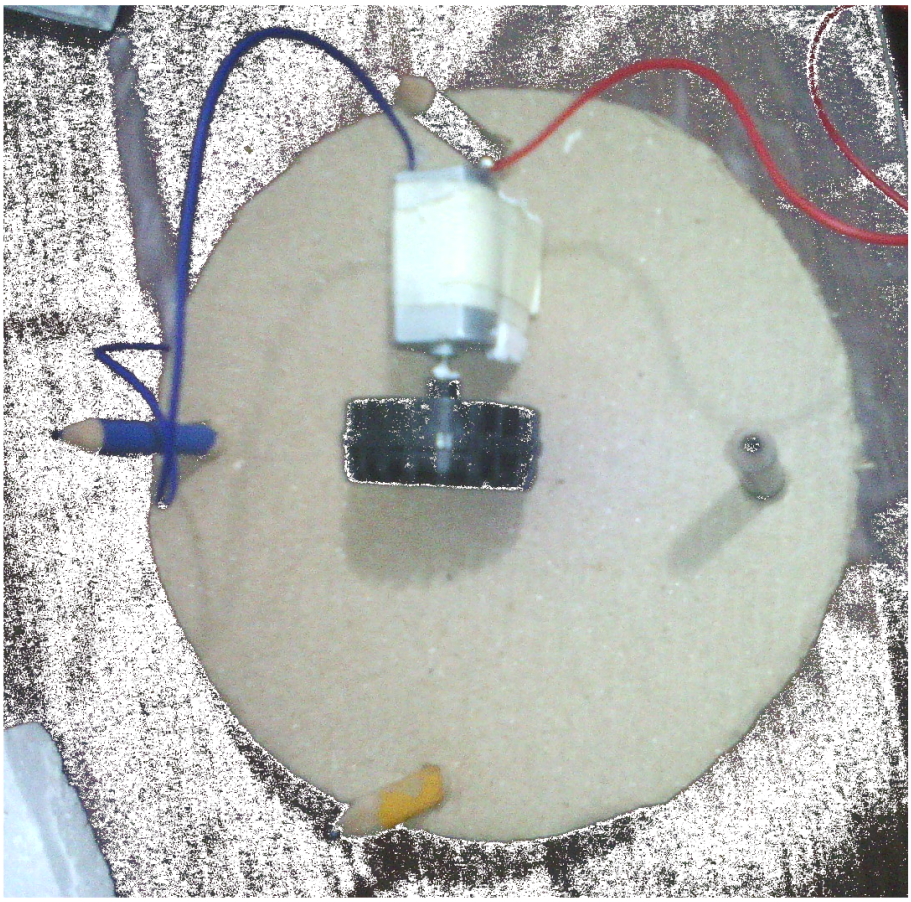

Figura 4.5: Robot 1

```
8  class Robot{
9          private:
10                 Button * button_;
11                 Motor * motor_;
12                 Timer * timer_;
13         public:
14                 Robot(Motor *, Timer *, Button *);
15                 void moveFordward(void);
16 };
17 #endif // ROBOT_H
```

La clase es simple, el constructor recibe apuntadores a instancias de las clases Motor, Timer y Button,[4] y las asigna a los apuntadores dentro de la clase robot. Veamos la implementación de los dos métodos.

```
1  #include "robot.h"
2
3  Robot::Robot(Motor * motor, Timer * timer, Button * button):
4          motor_{motor}, timer_{timer}, button_{button}
5  {
6  }
7  void Robot::moveFordward(void)
8  {
9          if(button_->pushed()){
10                 motor_->on();
11                 timer_->enable();
12                 while(timer_->getTime() <= 1000);
13                 motor_->off();
14         }
15 }
```

El método constructor muestra un concepto muy interesante, llamado dependency injection, dentro del diseño de software la inyección de dependencias es una forma de lograr desacoplar al módulo de sus dependencias, nosotros requerimos un motor, pero al solo pasar un apuntador o referencia, nos liberamos de conocer los detalles de su implementación, el robot solo sabe que existe un motor y que lo puede encender o apagar, pero no tiene idea de como esta conectado, de que tipo es, de corriente directa o alterna, tamaño o potencia, etc, lo mismo para el temporizador y botón.

[4] Realmente recibe apuntadores a instancias derivadas de estas.

Inicializados los apuntadores, los usamos para acceder a los métodos y creamos una sencilla rutina $moveFordward$, se explica por si sola, si el botón está presionado, enciende el motor por un segundo.

Motor

¿Y como se implementa el motor con interfaces?,

```
#ifndef MOTOR_H
#define MOTOR_H
class Motor{
        public:
                virtual void on(void) = 0;
                virtual void off(void) = 0;
};
#endif // MOTOR_H
```

La interfaz motor muestra dos métodos virtuales con una sintaxis un poco extraña, el termino $= 0$, es la forma que tiene *C++* de indicar que una clase es una interfaz, la palabra $virtual$, indican que las clases que hereden de ella pueden sobreescribir el método.

Ahora describirnos una clase con un motor particular de DC,

```
#ifndef MOTORDC_H
#define MOTORDC_H

#include "motor.h"

class MotorDc : public Motor{
        public:
                MotorDc(short const pin);
                virtual void on(void) override;
                virtual void off(void) override;
};
#endif // MOTORDC_H
```

Lo importante de esta clase es que hereda de la interfaz $Motor$ y que $override$ obliga a definir los métodos virtuales, independientemente del tipo de motor que se defina, seguirá siendo del tipo $Motor$, pues heredan de él y seguirán teniendo que implementar ambas funciones on y off. La implementación es:

```
#include "motorDc.h"
#include <avr/io.h>

MotorDc::MotorDc(short const pin)
{
        DDRD |= 1<<PD3;
}
void MotorDc::on(void)
{
        PORTD |= 1<<PD3;
}
void MotorDc::off(void)
{
        PORTD &= ~(1<<PD3);
}
```

Programar un microcontrolador puede resumirse a modificar sus registros[5],

Para implementar el motor modificamos dos registros, **DDRD** y **PORTD**, el primero Data direction register, nos permite configurar a el pin como entrada o salida, el método constructor configura al pin **PD3** como salida, el segundo registro nos permite indicar el nivel lógico, bajo o alto, 0 o 5 volts, respectivamente. Las funciones on y off modifican este registro, escribiendo nivel alto y bajo.

Para acceder a los registros es necesario incluir la biblioteca avr adecuada, en este caso $io.h$.

Botón

El módulo Button es igual a Motor, salvo que el pin **PD2**, se configura como entrada en **DDRD**, cuando un pin se elije como entrada, se obtiene una impedancia muy grande en el mismo, adicionalmente **PORTD** tiene un comportamiento diferente, escribir un uno lógico implica conectar una resistencia de Pull Up[6], activar esta resistencia tiene una gran utilidad, pues sin ella la alta impedancia en el pin provoca una gran sensibilidad al ruido electromagnético. La interfaz es la siguiente

```
#ifndef BUTTON_H
#define BUTTON_H
```

[5]Un registro es una locación de memoria en el microcontrolador, modificar los registros implica modificar el comportamiento del hardware

[6]Una resistencia conectada a Vcc

```
class Button{
        public:
                virtual bool pushed(void) = 0;
};
#endif // BUTTON_H
```

Tan solo cuenta con un método virtual que nos indica se el botón esta pulsado o no. La definición de un botón particular es:

```
#ifndef PUSH_H
#define PUSH_H

#include "button.h"

class Push : public Button{
        public:
                Push(short const pin);
                virtual bool pushed(void) override;
};
#endif // PUSH_H
```

y la implementación:

```
#include "push.h"
#include <avr/io.h>

Push::Push(short const pin)
{
        DDRD &= ~(1<<PD2);
        PORTD |= 1<<PD2;
}

bool Push::pushed(void)
{
        if((PIND&(1<<PD2)) == 0)
                return true;
        else
                return false;
}
```

Para leer el estado de un pin se consulta el registro **PIND**.

Timer

Timer es prácticamente idéntica en cuanto a la interfaz y descripción, por lo que solo muestro la implementación.

```
#include "timer_delay.h"
#include <util/delay.h>

void Timer_delay::enable(void)
{
}
unsigned long Timer_delay::getTime(void)
{
        _delay_ms(1000);
        return 1000;
}
```

Utilizo la función $_delay_ms$ provista por $delay.h$, para crear una pequeña pausa y después retorno 1000, es un poco hardcoded,[7] pero funciona y es probable que esto se borre en la próxima iteración.

Main

Finalmente enlazamos los módulos en la función main,

```
#include "robot.h"
#include "timer.h"
#include "button.h"
#include "motor.h"
#include "motorDc.h"
#include "timer_delay.h"
#include "push.h"


int main(void)
{
        MotorDC motor(1);
        Timer_delay timer;
        Push push(2);
```

[7] Hecho a mano, connotación negativa por ser una mala práctica

```
15          Robot robot(&motor,&timer, &push);
16
17          while(1){
18                  robot.moveFordward();
19          }
20
21          return 0;
22  }
```

Creamos los objetos de las instancias adecuadas y pasamos las direcciones al robot. El número que pasamos al motor y al botón push al construirlos indica el pin al que están conectados, en este momento no están implementados, no seguí mi propia regla y me adelanté, añadí algo que aun no necesito, seré más cuidadoso en el futuro.

4.5. Pruebas

Las pruebas son someras para este caso, tan solo nos interesa que el robot se mueva hacia adelante, no importa la velocidad ni la distancia, ni si quiera que lo haga en línea recta o de manera uniforme.

Las observaciones son las siguientes:

- El robot se mueve hacia adelante.
- El motor gira muy rápido y patina.
- La trayectoria es corta y no es rectilínea.
- Soporta poca carga.
- La carga eléctrica sube dramáticamente con carga mecánica.
- Las dimensiones de la llanta limitan la dimensión del motor.
- La nueva prioridad es añadir otro motor y realizar nuevamente el objetivo de mover hacia adelante.

Parece ser que perdí el tiempo, aunque el objetivo se logró, se encontraron muchas sensaciones negativas, entonces ¿no valió la pena la bala trazadora?, pienso que valió totalmente la pena, el tiempo de implementación fue relativamente corto, pero lo más importante fue la información que se obtuvo, ahora ya no tengo una hoja en blanco, si no una hoja de ruta, las sensaciones se reflejan en la refinación de objetivos.

4.6. Objetivos refinados

- Para el paciente.
  1. Mover el robot de un punto A a B.
     - Mover hacia adelante.
     - Mover hacia atrás.
     - Mover hacia adelante/atrás, en línea recta.
     - Mover hacia adelante una x distancia.
     - Mover hacia atrás una x distancia.
     - Mover hacia adelante/atrás y girar hacia la derecha/izquierda.
     - Girar x grados.
     - Mover sobre una trayectoria curva.
  2. Potencia suficiente para soportar la carga.
     - Suministro de corriente adecuado.
     - Tamaño y materiales de la llanta.
     - Motores con el tamaño y torque adecuado.
     - Acoplamiento con la base.
  3. Movimiento save.
     - Control de la velocidad.
     - Control de la aceleración.
     - Control diferenciado de ambos motores.
     - Generación y seguimiento de trayectorias.
  4. Rotación libre.
     - Basé rígida y pesada.
     - Acoplamiento encima la base.
- Para el médico.
  1. Interfaz de trayectorias.
- Para el desarrollador.
  1. Dominar un nuevo lenguaje de programación.
  2. Metodologías RUP y XP.

Capítulo 5

Doble propulsor. Segunda iteración

Es momento de trabajar en serio, en esta ocasión no avanzaré al siguiente objetivo si no se cumple primero el presente, para ello si lo requiero repetiré el proceso de manera continua tratando de mejorar el trabajo previo.

5.1. Requerimientos

El objetivo es casi el mismo solo adicionamos una especificación.

- Mover el robot hacia adelante.
- Utilizar un tren de propulsión de dos unidades.

5.2. Análisis

Nuevamente aprovechare el trabajo previo.

- Rediseñar el circuito, adicionando el segundo propulsor.
- Rediseñar la base acomodando ambos propulsores.
- Rediseñar el programa para dos propulsores.

5.3. Diseño

Estamos a tiempo para implementar estos cambios, tienen un gran impacto, pero la forma modular que adopté me facilita hacerlos sin mucha perdida de tiempo.

5.3.1. Arquitectura electrónica

Agregar un nuevo motor es una trivialidad, basta con repetir el modulo del motor, pero invirtiendo la polaridad, los motores estarán alineados y para que el robot avance, un motor debe girar en sentido dextrógiro y el otro en levógiro. El esquemático se muestra en la figura 5.1.

5.3.2. Arquitectura mecánica

La actualización es sencilla, se reemplazan dos postes por dos motores y se retira el motor central, figura 5.2.

5.3.3. Arquitectura de software

Robot requiere un modulo más abstracto que represente a los propulsores, propongo reemplazar Motor, por TrenDePotencia, el resto lo dejo igual por ahora, figura 5.3.

5.4. Implementación

5.4.1. Implementación electrónica y mecánica

No hay misterios cableamos y ensamblamos como indica el esquemático y plano.

5.4.2. Implementación de software

Esta parte requiere más cambios, creamos una nueva interfaz, llamada $PowerTrain$ que sustituye a $Motor$ pues esta última ya no nos sirve para esta etapa, implementamos un tren de potencia $Differential$, tan solo contiene dos referencias a dos motores, izquierdo y derecho y la respectivas implementaciones de las funciones virtuales de on y off.

```
#include "differential.h"
Differential::Differential(Motor * left, Motor * right)
```

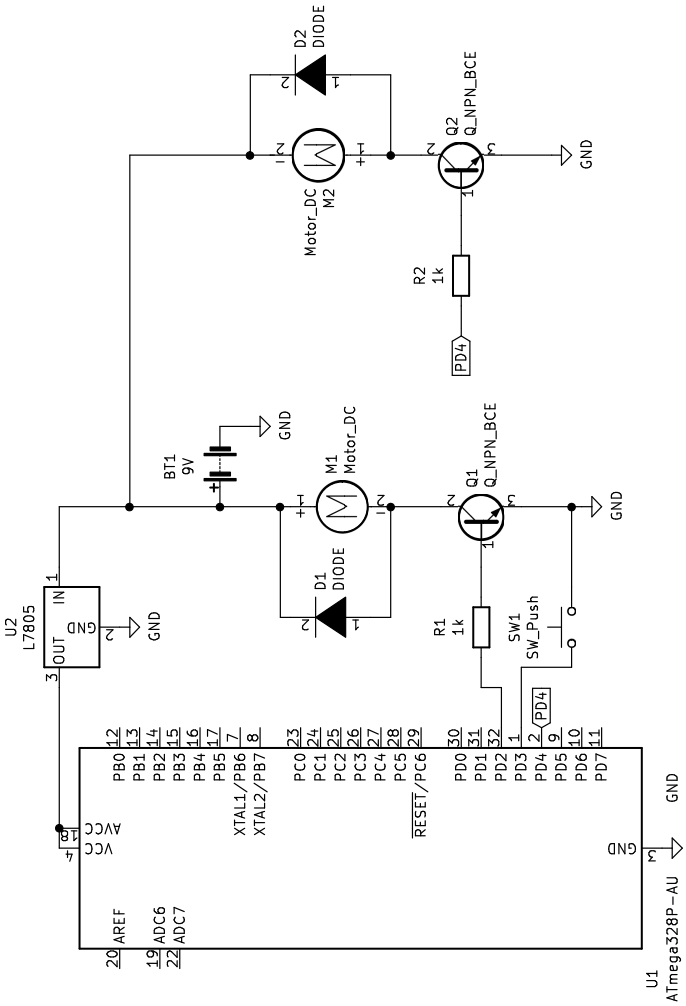

Figura 5.1: Esquemático 2

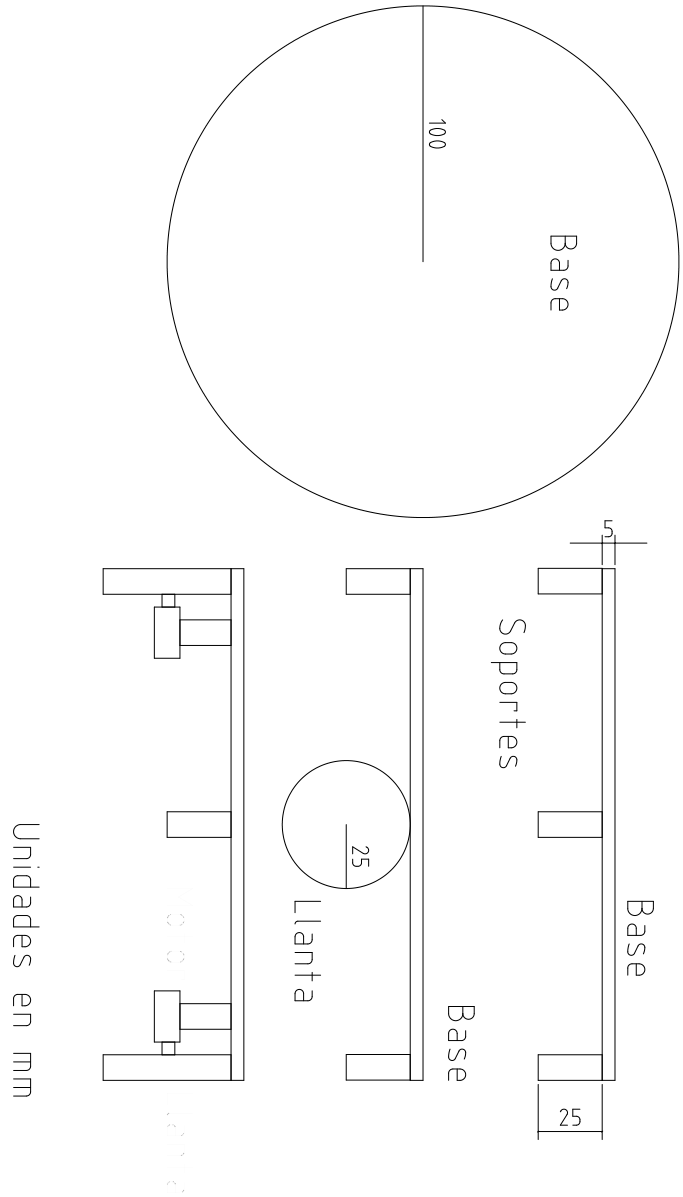

Figura 5.2: Plano 2

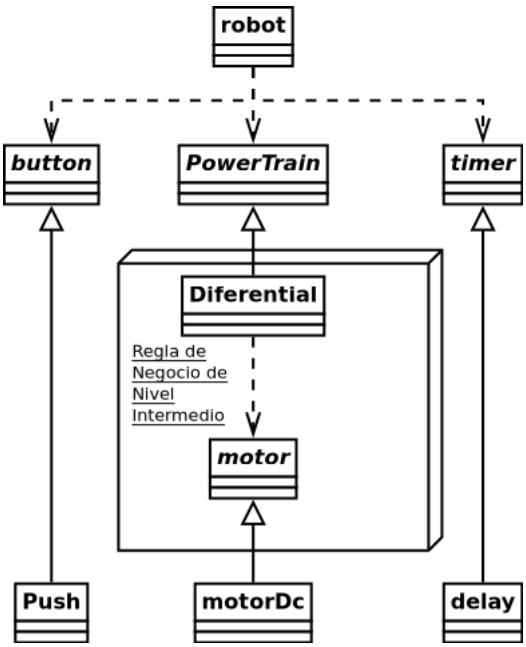

Figura 5.3: Arquitectura de software 2

```
3          : rigth_{right}, left_{left}
4  {
5  }
6  void Differential::on(void)
7  {
8          rigth_->on();
9          left_->on();
10 }
11 void Differential::off(void)
12 {
13         rigth_->off();
14         left_->off();
15 }
```

Firmware

La definición de firmware ha cambiado a lo largo del tiempo, hoy en día al firmware se le identifica como el software que interactúa directamente con el hardware, en términos de microcontroladores, el firmware es el software que interactúa directamente con los registros. En nuestro caso tenemos firmware en la implementación de $MotorDc$ y en $Push$.

Probablemente la forma correcta de escribir firmware es escribir menos firmware, por naturaleza cambia junto con el hardware, es decir se vuelve obsoleto con los cambios de hardware, prefiero aislarlo en una pequeña biblioteca y llamarlo desde $Motor$, en el diagrama de arquitectura figura 5.3, no lo incluí pues la idea me surgió después de diseñarlo, pero no importa, pues la metodología me permite agregarlo posteriormente. La implementación de $Motor$ es:

```
1  #include "motorDc.h"
2  #include "gpio.h"
3
4  MotorDc::MotorDc(short const pin) : pin_{pin}
5  {
6          gpio_setDirection(pin_, OUTPUT);
7  }
8  void MotorDc::on(void)
9  {
10         gpio_setLevel(pin_, HIGH);
11 }
```

```
12 void MotorDc::off(void)
13 {
14         gpio_setLevel(pin_, LOW);
15 }
```

Ahora $MotorDC$ ya no depende de los registros, la responsabilidad se pasa a dos funciones $gpio_setLevel$ y $gpio_setDirection$, no es el objetivo atiborrar el presente documento de código, por lo que prefiero no presentar la implementación de dichas funciones pues no forman parte del robot, son simples utilidades, el código completo está disponible en el repositorio.

5.5. Pruebas

Las expectativas son altas, pues la implementación y diseño fueron fáciles, increíblemente los resultados son un fracaso. Ensamblado, actualizado y puesto en marcha obtengo las siguientes observaciones:

- Avanza un par de milímetros y se detiene.
- En el aire las ruedas giran aveces.

Repitiendo varias veces el experimento me doy cuenta que cuando presiono el botón el microcontrolador se reinicia.

En el pasado utilizamos solo un motor y funcionaba, sin embargo con dos motores la carga es demasiada para la fuente y por un instante la caída de tensión es suficientemente alta como para que el microcontrolador se apague.

Debo repetir todos los pasos y ajustar en donde considere necesario.

5.6. Segunda iteración

5.6.1. Requerimientos y Análisis

Sin cambios.

5.6.2. Diseño

Diseño electrónico

El problema es claramente eléctrico, la primera reacción es agregar un capacitor grande para suavizar el arranque del motor, sin embargo no me da buenas sensacio-

nes, aveces funciona y en otras se sigue apagando, la solución temporal más sensata es separar la fuente de los motores de la fuente del microcontrolador.

Por comodidad decido cambiar el tipo de pulsador por uno normalmente apagado, los cambios se ven reflejados en el esquemático de la figura 5.4.

Diseño mecánico

Sin cambios.

Diseño del programa

Se agregan los cambios faltantes en la anterior iteración, figura 5.5.

5.6.3. Implementación

El único cambio es agregar una batería como indica el esquemático, figura 5.4.

5.6.4. Pruebas

¡Éxito!, el robot se mueve hacia adelante, observaciones:

- Recorre alrededor de 1 metro.
- El movimiento es siempre con tendencia a la derecha.
- El microcontrolador ya no se apaga.

No requiero pruebas más exhaustivas para este requisito.

5.7. Objetivos refinados

- Para el paciente.
    1. Mover el robot de un punto A a B.
        - ~~Mover hacia adelante~~.
        - Mover hacia atrás.
        - Mover hacia adelante/atrás, en línea recta.
        - Mover hacia adelante una x distancia.
        - Mover hacia atrás una x distancia.

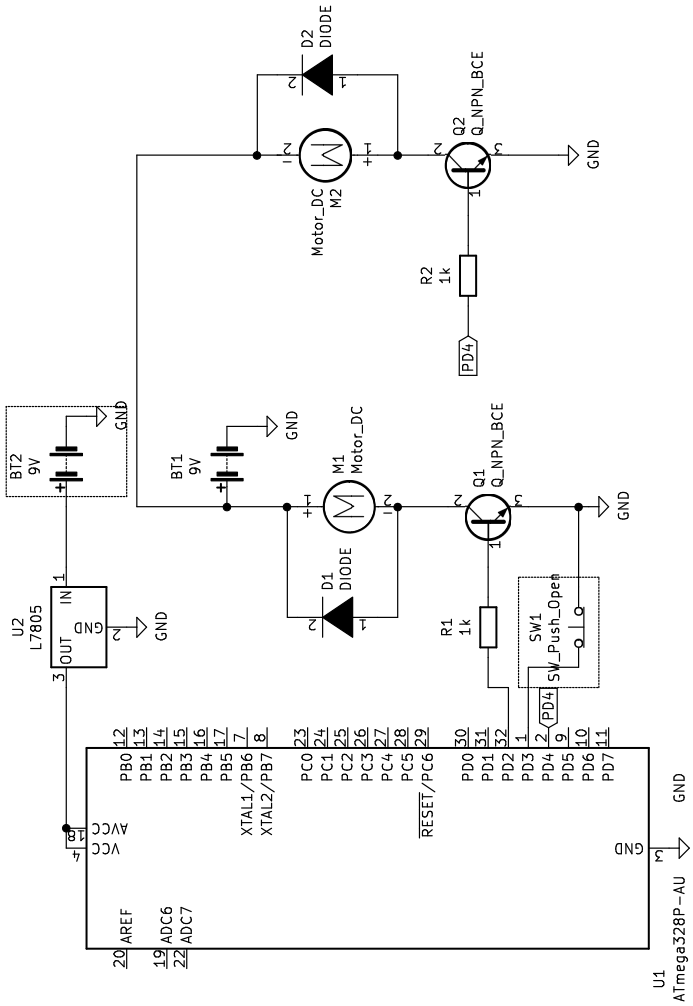

Figura 5.4: Esquemático 2.2

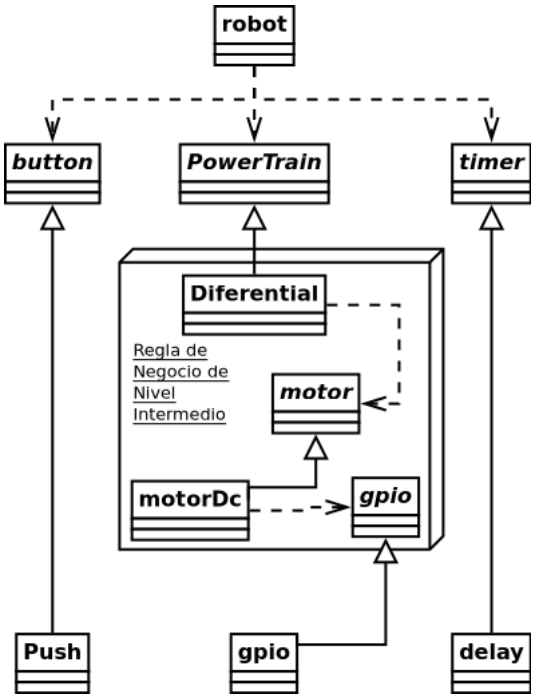

Figura 5.5: Arquitectura de software 2.2

    - Mover hacia adelante/atrás y girar hacia la derecha/izquierda.
    - Girar x grados.
    - Mover sobre una trayectoria curva.
  2. Potencia suficiente para soportar la carga.
    - **Fuente de controlador**.
    - **Fuente de potencia**.
    - Tamaño y materiales de la llanta.
    - Motores con el tamaño y torque adecuado.
    - Acoplamiento con la base.
  3. Movimiento save.
    - Control de la velocidad.
    - Control de la aceleración.
    - Control diferenciado de ambos motores.
    - Generación y seguimiento de trayectorias.
  4. Rotación libre.
    - Basé rígida y pesada.
    - Acoplamiento encima la base.

- Para el médico.
  1. Interfaz de trayectorias.
- Para el desarrollador.
  1. Dominar un nuevo lenguaje de programación.
  2. Metodologías RUP y XP.

Capítulo 6

Volviendo al pasado, tercera iteración

Continuando con el desarrollo, implementado el movimiento hacia adelante, el siguiente objetivo natural es ir hacia atrás.

6.1. Requerimientos

Una de las trayectorias más comunes que requiere el paciente es una simple línea recta, de atrás hacia adelante y re regreso.

- Mover el robot hacia atrás.

6.2. Análisis

Aunque la idea es simple, el cumplimiento de este objetivo nos revela grandes agregados a el trabajo previo:

- Invertir la corriente en los motores.
- Añadir la funcionalidad reversa en el programa.

6.3. Diseño

Las partes implicadas son la parte eléctrica y de programación.

6.3.1. Diseño electrónico

Hasta ahora el transistor ha sido suficiente, pero ahora requiero una forma de invertir la corriente que pasa por el motor, una forma de hacerlo es con un circuito tipo puente, con un arreglo de seis transistores, cuatro npn y dos pnp, como se indica en la figura 6.1, cuando PINB se pone en nivel alto, provoca que Q3 y Q7 conduzcan, cuando Q3 conduce corriente, provoca que Q4 conduzca, con Q4 y Q7 conduciendo, la corriente puede fluir a través del motor, si es PINA el que se enciende entonces la corriente fluye por el motor de manera inversa, por supuesto si PINA se pone en alto entonces PINB debe estar en bajo y viceversa, de los contrario abría un corto circuito que probablemente dañaría los transistores. Sin embargo hay

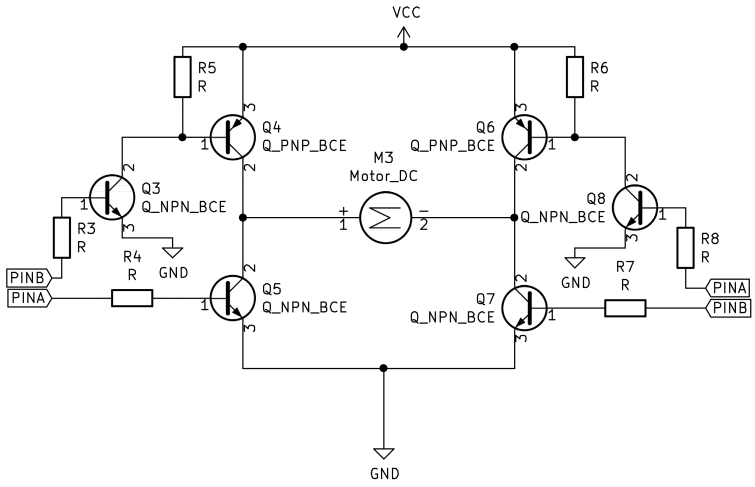

Figura 6.1: Puente para motor de DC

otra opción más simple, se puede utilizar un circuito integrado que ya contenga esta funcionalidad, los circuitos de este tipo son llamados de tipo puente. Uno muy utilizado es el *L298*[5], en especifico este circuito integrado tiene la capacidad de manejar a dos motores, con una potencia total de $25[W]$ y una corriente continua máxima de $2[A]$ por cada motor, es suficiente por ahora pero tengo mis dudas para el futuro, a ojo de buen cubero pienso que requiero al menos $20[W]$ por cada motor, llegado ese momento decidiré que hacer con el circuito integrado.[1] Regresando al L298, nos permite controlar la dirección de rotación de los motores y mediante una técnica de conmutación también podemos controlar su velocidad. En la figura 6.2 se muestra el esquemático.[2] El circuito integrado esta dividido en dos canales A y B, uno para cada motor, para lograr que el motor gire en la dirección que decidamos debemos establecer las señales correspondientes a los pines de cada canal, para el canal A los pines correspondientes son:

Pin 1 Sensor de corriente, para medir la corriente del motor.

Pines 2 y 3 Salida de potencia, aquí se conectan las terminales del motor.

Pines 5 y 7 Entradas lógicas de control, estableciendo 1 y 0 respectivamente el motor gira a la derecha, con 0 y 1 gira en sentido contrario, con 0 y 0 o con 1 y 1 el motor se detiene.

Pin 6 Habilitar, establecer un valor lógico 1, habilita la salida de potencia.

Por lo tanto requerimos al menos tres pines de nuestro microcontrolador para controlar cada uno de los motores, tenemos muchos pines disponibles, no es un problema, no requerimos medir la corriente así que los pines 1 y 15 van directos a tierra. El esquemático se ha simplificado mucho pese a que añadí funcionalidad, estoy satisfecho.

6.3.2. Diseño de programa

La arquitectura es la misma, los cambios se dan en el modulo $MotorDc$, se requiere que cada motor utilice tres pines.

[1] ¿Por qué no usar un circuito integrado más poderoso de una vez?, no hay ninguna razón técnica, es el que tengo disponible en este instante.

[2] El L298 requiere dos fuentes de voltaje una para entregar potencia a los motores y otra para el control.

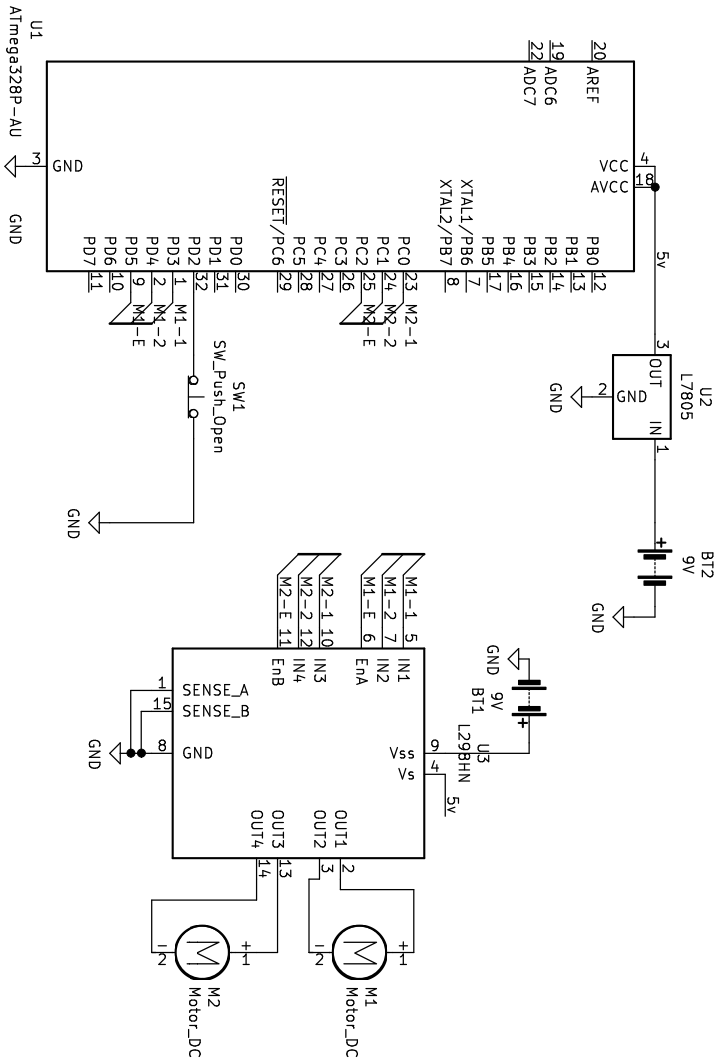

Figura 6.2: Esquemático 3

6.4. Implementación

La implementación eléctrica y mecánica como es costumbre consiste en seguir los esquemáticos y planos, los cambios en esta etapa se dan en el software.

Modificamos la clase $MotorDc$ para contener los tres pines y para añadir un nuevo comportamiento, ahora los motores pueden ir hacia adelante o hacia atrás.[3]

```
class MotorDc : public Motor{
        private:
                short pin_enable_;
                short pin_input1_;
                short pin_input2_;
        public:
                MotorDc(short const pin_enable,
                        short const pin_input1,
                        short const pin_input2);
                virtual void forward(void) override;
                virtual void backward(void) override;
                virtual void off(void) override;
};
```

la implementación es igualmente simple, se configuran los tres pines para indicar el comportamiento ($pin_enable_$, $pin_input1_$, $pin_input2_$), se reemplaza el método on por $fordward$ y $backward$. La implementación es la siguiente:

```
1  #include "motorDc.h"
2  #include "gpio.h"
3
4  MotorDc::MotorDc(short const pin_enable,
5                   short const pin_input1,
6                   short const pin_input2) :
7                   pin_enable_{pin_enable},
8                   pin_input1_{pin_input1},
9                   pin_input2_{pin_input2}
10 {
11         gpio_setDirection(pin_enable_, OUTPUT);
12         gpio_setDirection(pin_input1_, OUTPUT);
13         gpio_setDirection(pin_input2_, OUTPUT);
14 }
```

[3] Igualmente se modifican las declaraciones de los métodos virtuales en la interfaz $Motor$

```
15 void MotorDc::forward(void)
16 {
17         gpio_setLevel(pin_enable_, HIGH);
18         gpio_setLevel(pin_input1_, HIGH);
19         gpio_setLevel(pin_input2_, LOW);
20 }
21
22 void MotorDc::backward(void)
23 {
24         gpio_setLevel(pin_enable_, HIGH);
25         gpio_setLevel(pin_input1_, LOW);
26         gpio_setLevel(pin_input2_, HIGH);
27 }
28
29 void MotorDc::off(void)
30 {
31         gpio_setLevel(pin_enable_, LOW);
32         gpio_setLevel(pin_input1_, LOW);
33         gpio_setLevel(pin_input2_, LOW);
34 }
```

Ahora que cambiamos la interfaz de $Motor$, el módulo $differential$ ya no entiende a los motores hay que modificarlo también,[4] $differential$ tiene dos funciones, las modifico igual que $motor$,[5] recordemos que los motores deben girar en direcciones opuestas para hacer el desplazamiento.

```
1  #ifndef DIFFERENTIAL_H
2  #define DIFFERENTIAL_H
3
4  #include "power_train.h"
5  #include "motor.h"
6
7  class Differential : public PowerTrain{
8          private:
9                  Motor *left_;
10                 Motor *rigth_;
```

[4]Igualmente se modifica la interfaz PowerTrain

[5]En el futuro differential tendrá funciones diferentes a Motor, es pura coincidencia que ahora sean iguales.

```
11          public:
12                  Differential(Motor *, Motor *);
13                  virtual void forward(void) override;
14                  virtual void backward(void) override;
15                  virtual void off(void) override;
16  };
17
18  #endif // DIFFERENTIAL_H
```

La implementación se muestra a continuación:

```
1   #include "differential.h"
2   Differential::Differential(Motor * left, Motor * right) :
3                 rigth_{right}, left_{left}
4   {
5   }
6
7   void Differential::forward(void)
8   {
9           left_->forward();
10          rigth_->backward();
11  }
12
13  void Differential::backward(void)
14  {
15          left_->backward();
16          rigth_->forward();
17  }
18
19  void Differential::off(void)
20  {
21          rigth_->off();
22          left_->off();
23  }
```

Invito al lector a observar lo simple y expresivo que es el código. Nuevamente cambiamos una interfaz, ahora todos los módulos que dependen de ella deben ser actualizados para que entiendan las nuevas funcionalidades, en este caso $robot$. Actualmente robot solo tiene un método, $moveForward$, agregar $moveBackward$ es suficiente por ahora:

```
#include "robot.h"

Robot::Robot(PowerTrain * train, Timer * timer):
        train_{train}, timer_{timer}
{
}
void Robot::moveForward(void)
{
        train_->forward();
        timer_->enable();
        while(timer_->getTime() < 1000);
        train_->off();
}
void Robot::moveBackward(void)
{
        train_->backward();
        timer_->enable();
        while(timer_->getTime() < 1000);
        train_->off();
}
```

Decido extraer el botón y llevarlo a main por comodidad, el código puede mejorarse pero tengo la sensación de que se modificara en el siguiente hito y no vale la pena la optimización en este instante. Finalmente el nuevo main queda así:

```
#include "robot.h"
#include "motorDc.h"
#include "differential.h"
#include "timer_delay.h"
#include "push.h"


int main(void)
{
        MotorDc left(9, 1, 2);
        MotorDc right(25, 23, 24);
        Differential train(&left, &right);
        Timer_delay timer;
        Robot robot(&train, &timer);
        Push push(32);
```

```
        while(1){
                if(push.pushed()){
                        robot.moveForward();
                        robot.moveBackward();
                }
        }
        return 0;
}
```

6.5. Pruebas

Lo único que nos interesa es que el robot se mueva hacia atrás, puesto en marcha observo lo siguiente.

- El robot avanza y al pasar un segundo cambia bruscamente hacia atrás.
- El freno abrupto provoca una rotación indeseable.
- Avanza más de lo que retrocede.
- Se percibe una menor potencia que cuando usaba un transistor.

Se da por cumplido el objetivo y se refinan los siguientes para corregir las sensaciones negativas que se han encontrado.

6.6. Objetivos refinados

- Para el paciente.
    1. Mover el robot de un punto A a B.
        - ~~Mover hacia adelante~~.
        - ~~Mover hacia atrás~~.
        - Mover hacia adelante/atrás, en línea recta.
        - Mover hacia adelante una x distancia.
        - Mover hacia atrás una x distancia.
        - Mover hacia adelante/atrás y girar hacia la derecha/izquierda.
        - Girar x grados.

    - Mover sobre una trayectoria curva.
  2. Potencia suficiente para soportar la carga.
    - Fuente de controlador.
    - Fuente de potencia.
    - **Selección del circuito de potencia**.
    - Tamaño y materiales de la llanta.
    - Motores con el tamaño y torque adecuado.
    - Acoplamiento con la base.
  3. Movimiento save.
    - Control de la velocidad.
    - Control de la aceleración.
    - Control diferenciado de ambos motores.
    - Generación y seguimiento de trayectorias.
  4. Rotación libre.
    - Basé rígida y pesada.
    - Acoplamiento encima la base.

- Para el médico.
  1. Interfaz de trayectorias.
- Para el desarrollador.
  1. Dominar un nuevo lenguaje de programación.
  2. Metodologías RUP y XP.

Capítulo 7

Moverse en línea recta

Me interesa crear una trayectoria, para lograrlo requiero que el robot se mueva en línea recta cuando yo lo indique. Actualmente el robot se desvía de la trayectoria recta, la razón es la diferencia de potencia en los motores. Para corregir la desviación propongo reducir la potencia del motor "dominante", e igualarla con el motor más débil.

Para reducir la potencia se requiere reducir la corriente que pasa por el motor, una resistencia en serie bastaría, para calcular el valor de la resistencia se tendría que hacer un experimento probando diferentes resistencias y encontrar la óptima por prueba y error.

Otra forma de hacerlo es reduciendo la tensión por medio de una onda pwm.[1] El circuito integrado que utilizamos para controlar la potencia de los motores tiene un pin para habilitar y deshabilitar la corriente en los motores, cuando el pin esta habilitado la tensión en los motores es de $9[V]$,[2] por el contrario, cuando el pin esta deshabilitado la tensión es $0[V]$.

Una onda pwm se comporta como un interruptor que se enciende y apaga constantemente a un ritmo determinado, al ritmo se le llama frecuencia, a la relación que existe entre el tiempo de encendido y apagado se le conoce como ciclo de trabajo o como ancho de pulso figura 7.1. Como consecuencia de una onda pwm en el pin de habilitación, la tensión se reduce proporcionalmente al ciclo de trabajo, para un ciclo de trabajo del cincuenta por ciento, la tensión se reduce en esa misma proporción.

[1] Pulsed Wave Modulated.

[2] En realidad es de $7[V]$, este circuito tiene una caída de tensión debido a los transistores con los que esta construido.

Si creamos una onda pwm de frecuencia igual a diez segundos y con un ciclo de trabajo del cincuenta por ciento, ¿tendremos la mitad de la tensión durante los diez segundos?, por supuesto que no. El promedio de la tensión es la mitad, pero físicamente no vemos ese comportamiento con frecuencias tan bajas, para apreciarlo se requiere frecuencias relativamente altas del orden de los kilohertz, si en lugar de hacer una pwm de diez segundos la hacemos de un milisegundo y de duración total de diez segundos, entonces si percibiremos una caída de tensión a la mitad.

7.1. Requerimientos

Para poder realizar algunas trayectorias es necesario que el robot se mueva de forma rectilínea.

7.2. Análisis

Para la situación actual basta con reducir la potencia en el motor más potente.

- Rediseñar la interfaz de motor para indicar la intensidad de la acción al motor.
- Implementar cambio de intensidad con una onda pwm.

7.3. Diseño

El diseño mecánico se mantiene igual que la iteración anterior. El diseño electrónico se modifica mínimamente se cambian los dos pines habilitadores de los motores, el pin habilitador del motor izquierdo se cambia al pin 10 y el del derecho al pin 9, no se muestra el diagrama.

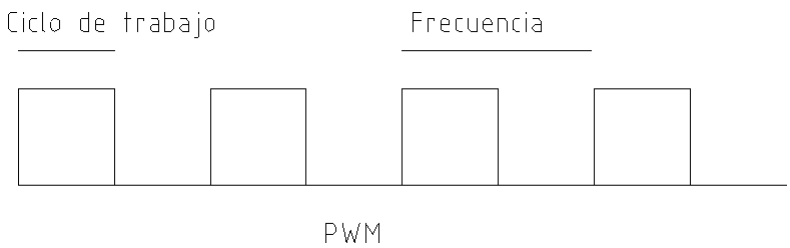

Figura 7.1: PWM

7.3.1. Diseño del programa

En este instante tengo la disyuntiva de usar *concurrencia* o no. Hasta ahora el programa se ejecuta en un solo proceso, instrucción por instrucción y el flujo es guiado por las condiciones que he impuesto, añadir concurrencia implica añadir un segundo proceso, que ejecute parte del programa en pseudo paralelo,[3] añadir un segundo proceso simplifica el desarrollo, pero decrementa la portabilidad y peor aun la mantenibilidad.

¿Para qué necesitamos concurrencia?, el atmega328p contiene hardware para realizar señales pwm via un timer. Un timer es un periférico del microcontrolador cuyas funcionalidades están especializadas en manejo de ciclos del reloj. La operación del timer implica un proceso concurrente.

Actualmente utilizo la función delay, esta función bloquea el flujo, realizar la pwm por hardware implica poder seguir usando delay, realizar la pwm por software por el contrario me impide seguir utilizando delay, obligándome a diseñar un nuevo reloj que no bloqueé el flujo. Además el código es totalmente dependiente de los registros del dispositivo.

La metodología XP recomienda hacer cambios pequeños e incrementales, por este motivo decido agregar la concurrencia utilizando las capacidades del hardware asumiendo las desventajas que esta decisión conlleva.

Timer 0

El atmega328p tiene tres timer's o temporizadores,

- Timer 0, 8 bits.
- Timer 1, 16 bits.
- Timer 2. 8 bits.

Los tres temporizadores tienen el hardware necesario para hacer pwm, el segundo tiene mayor flexibilidad, pero es más difícil de configurar, elijo el timer 0.

La generación de ondas pwm con un timer es sencillo, involucra tan solo cuatro elementos,

- Un reloj para el timer.
- Un registro contador TCNT.

[3] Realmente no se ejecuta en paralelo, simplemente se ejecuta un proceso durante un tiempo y después se intercambia por el otro proceso.

- Un registro comparador OCR.
- Un pin donde se genera la señal.

Se indica un valor en el registro de comparación ORC, el pin inicia en nivel bajo, se enciende el reloj, con cada ciclo de reloj el registro $TCNT$ incrementa su valor en uno; cuando han pasado suficientes ciclos de reloj y el valor de $TCNT$ alcanza al valor de ORC, automáticamente el nivel del pin es cambiado a alto, $TCNT$ sigue aumentando con cada ciclo de reloj hasta llegar a 255, con un ciclo de reloj adicional $TCNT0$ cambia a 0, a esto se le conoce como overflow, $TCNT0$ vuelve a ser menor que OCR, automáticamente el pin de señal cambia a nivel bajo.

El timer tiene dos modos de ondas pwm,

- Fast.
- Phase correct.

El modo fast, nos permite una mayor frecuencia a cambio de un error en el ancho de pulso, no es un error que considere relevante para el propósito de controlar el motor, el modo phase correct no tiene este error pero las frecuencias son más bajas.

Para evitar problemas de conmutación seleccionare una frecuencia no tan alta, pues al circuito integrado que controla a los motores L298, no le gustan las frecuencias altas, otra razón más para cambiarlo. Para el reloj de $16[MHz]$ las frecuencias permitidas son:

Modo fast

1. $62.5[kHz]$
2. $7.8[kHz]$
3. $976[Hz]$
4. $244[Hz]$
5. $61[Hz]$

Modo phase correct

1. $31.3[kHz]$
2. $3.9[kHz]$
3. $490[Hz]$
4. $122[Hz]$
5. $30[Hz]$

La frecuencia más interesante sin duda es la de $31.3[kHz]$, esta justo por arriba de la frecuencia audible, sin embargo la hoja de datos del circuito integrado L298 nos recomienda una frecuencia de $25[kHz]$, por ahora elegiré una frecuencia más baja y si cambio el integrado aumentaré la frecuencia, $7.8[kHz]$ es la opción elegida.

Desafortunadamente los pines generadores están fijos y no son configurables, para el timer 0 son solo dos, los pines 9 y 10, suficientes pues solo tenemos dos motores.

A continuación una lista de los registros que se requieren modificar y lo que se modificar en ellos sin entrar en detalles.

TCCR0A Registro A de control del timer 0, habilitamos los pines 9 y 10 para la generación de ondas e indicamos que el timer se utilizara en modo generación fast pwm.

TCCR0B Registro B de control de timer 0, habilitamos el counter $TCNT0$ y seleccionamos la frecuencia.

TCNT0 Counter register, incrementa su valor con cada ciclo del reloj de timer.

OCR0A Output compare register A, aquí indicamos el valor al que TCNT0 tiene que llegar para cambiar el valor del pin que genera la onda(pin 10).

OCR0B Output compare register B, aquí indicamos el valor al que TCNT0 tiene que llegar para cambiar el valor del pin que genera la onda(pin 9).

Hay registros adicionales para mayores funcionalidades, nos quedamos con esto por ahora y si lo necesitamos los utilizaremos.

Arquitectura de software

La modificación a la arquitectura se muestra en la figura 7.2,

Código

Desarrollamos un nuevo módulo que implemente los cambios en los registros para generar una pwm en los pines 9 y 10, la declaración del modulo es la siguiente:

```
void pwm_enablePin(short const pin);
void pwm_disablePin(short const pin);
void pwm_enableTimer(void);
void pwm_disableTimer(void);
void pwm_setWorkCycle(short const pin, short wc);
```

La implementación es igual de simple:

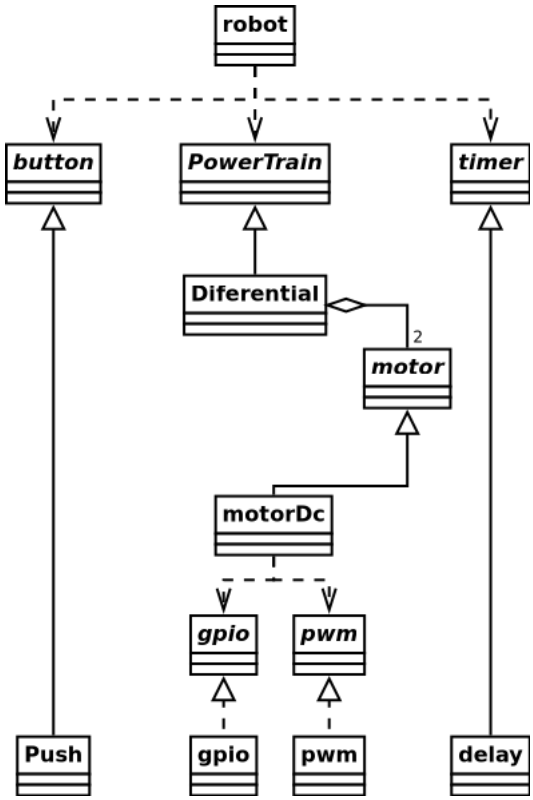

Figura 7.2: Arquitectura de software 4

```
#include "pwm.h"
#include <avr/io.h>

void pwm_enablePin(short const pin)
{
        if(pin == 10)
                TCCR0A |= 1<<COM0A1|1<<COM0A0;
        else if(pin == 9)
                TCCR0A |= 1<<COM0B1|1<<COM0B0;
        //enable Fast mode
        TCCR0A |= 1<<WGM01|1<<WGM00;
}

void pwm_disablePin(short const pin)
{
        if(pin == 9)
                TCCR0A &= ~(1<<COM0B1|1<<COM0B0);
        else if(pin == 10)
                TCCR0A &= ~(1<<COM0A1|1<<COM0A0);
}

void pwm_enableTimer(void)
{
        TCCR0B |= 1<<CS01;
}

void pwm_disableTimer(void)
{
        TCCR0B &= ~(1<<CS01);
}

void pwm_setWorkCycle(short const pin, short wc)
{
        if(wc > 100)
                wc = 100;
        if(wc < 0)
                wc = 0;
        if(pin == 10)
                OCR0A = wc/100.0*255;
```

```
        else if(pin == 9)
                OCR0B = wc/100.0*255;
}
```

Las funciones únicamente cambian el valor de los registros.

Motor debe cambiar su interfaz, deberá recibir como argumento el porcentaje de potencia que queremos que desarrolle.

```
class Motor{
        public:
                virtual void forward(short const power) = 0;
                virtual void backward(short const power) = 0;
                virtual void off(void) = 0;
};
```

Basta con un include en $MotorDc$ para obtener las nuevas funcionalidades de pwm.

La implementación en el módulo $MotorDC$ es la siguiente:

```
1  #include "motorDc.h"
2  #include "gpio.h"
3  #include "pwm.h"
4
5  MotorDc::MotorDc(short const pin_enable,
6          short const pin_input1,
7          short const pin_input2) :
8          pin_enable_{pin_enable},
9          pin_input1_{pin_input1},
10         pin_input2_{pin_input2}
11 {
12         gpio_setDirection(pin_enable_, OUTPUT);
13         gpio_setDirection(pin_input1_, OUTPUT);
14         gpio_setDirection(pin_input2_, OUTPUT);
15         pwm_enableTimer();
16 }
17
18 void MotorDc::forward(short const power)
19 {
20         gpio_setLevel(pin_input1_, HIGH);
21         gpio_setLevel(pin_input2_, LOW);
22         pwm_setWorkCycle(pin_enable_, power);
```

```
23          pwm_enablePin(pin_enable_);
24  }
25
26  void MotorDc::backward(short const power)
27  {
28          gpio_setLevel(pin_input1_, LOW);
29          gpio_setLevel(pin_input2_, HIGH);
30          pwm_setWorkCycle(pin_enable_, power);
31          pwm_enablePin(pin_enable_);
32  }
33
34  void MotorDc::off(void)
35  {
36          gpio_setLevel(pin_input1_, LOW);
37          gpio_setLevel(pin_input2_, LOW);
38          pwm_disablePin(pin_enable_);
39  }
```

Es el pin pin_enable quien actúa con la interfaz pwm.

Para finalizar, el módulo diferencial debe ser actualizado pues la interfaz $motor$ fue modificada, ahora podemos especificar la potencia a cada motor, por ejemplo para un 70-100, basta con:

```
1   #include "differential.h"
2   Differential::Differential(Motor * left, Motor * right) :
3                 rigth_{right}, left_{left}
4   {
5   }
6
7   void Differential::forward(void)
8   {
9           left_->forward(70);
10          rigth_->backward(100);
11  }
12
13  void Differential::backward(void)
14  {
15          left_->backward(70);
16          rigth_->forward(100);
17  }
```

```
18
19 void Differential::off(void)
20 {
21         rigth_->off();
22         left_->off();
23 }
```

7.4. Pruebas

Descargado el nuevo código sobre el microcontrolador, se realizan varios eventos, donde por prueba y error se trata de corregir la desviación cambiando la onda pwm de cada motor.

Las sensaciones son frustrantes, no es posible obtener una línea recta con solo cambiar la potencia de los motores.

- Encontrar los valores de potencia adecuados es difícil.
- Si la superficie cambia, los valores también cambian.
- Para cada valor se tiene que descargar el programa.

Fui demasiado optimista, por más ajustes que haga no me sera posible obtener la línea recta, prefiero dar por concluido esté hito, pero sin tachar el objetivo y en su lugar agregar y tachar, potencia diferenciada, aprovecharemos la onda pwm para realizar un controlador por asignación de polos que definitivamente nos hará cumplir el objetivo de ir en línea recta.

7.5. Objetivos refinados

- Para el paciente.
    1. Mover el robot de un punto A a B.
        - ~~Mover hacia adelante~~.
        - ~~Mover hacia atrás~~.
        - ~~Potencia diferenciada~~.
        - Mover hacia adelante/atrás, en línea recta.
        - Mover hacia adelante una x distancia.
        - Mover hacia atrás una x distancia.

    - Mover hacia adelante/atrás y girar hacia la derecha/izquierda.
    - Girar x grados.
    - Mover sobre una trayectoria curva.
  2. Potencia suficiente para soportar la carga.
    - Fuente de controlador.
    - Fuente de potencia.
    - Selección del circuito de potencia.
    - Tamaño y materiales de la llanta.
    - Motores con el tamaño y torque adecuado.
    - Acoplamiento con la base.
  3. Movimiento save.
    - Control de la velocidad.
    - Control de la aceleración.
    - Control diferenciado de ambos motores.
    - Generación y seguimiento de trayectorias.
  4. Rotación libre.
    - Basé rígida y pesada.
    - Acoplamiento encima la base.

- Para el médico.
  1. Interfaz de trayectorias.
- Para el desarrollador.
  1. Dominar un nuevo lenguaje de programación.
  2. Metodologías RUP y XP.

Capítulo 8

Un nuevo giro al proyecto

Bibliografía

[1] *8-bit AVR Microcontroller with 32K Bytes In-SystemProgrammable Flash*. atmega328p. 7810D-AVR-01/15. Microchip. 2015. URL: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.

[2] *Amplifier Transistors NPN Silicon*. P2N2222A. Rev. 7. ON Semiconductors. 2013. URL: https://www.onsemi.com/pdf/datasheet/p2n2222a-d.pdf.

[3] Kent Beck. *Test Driven Development: By Example*. 1ed. Addison-Wesley Professional, 2002.

[4] Andy Hunt Dave Thomas. *The pragmatic programmer: your journey to mastery*. 20ed Anniversary Edition. Addison-Wesley, 2020.

[5] *DUAL FULL-BRIDGE DRIVER*. L298. Rev. 1. STMicroelectronics. 2000. URL: https://www.st.com/resource/en/datasheet/l298.pdf.

[6] Grady Booch Et al. *Object-oriented analysis and design with applications*. 3ed. Addison-Wesley, 2007.

[7] Ivar Jacobson Et al. *The Essentials of Modern Software Engineering: Free the Practices from the Method Prisons!* 1ed. ACM Books, 2019.

[8] Robe Cecil Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. 1ed. Prentice Hall, 2018.